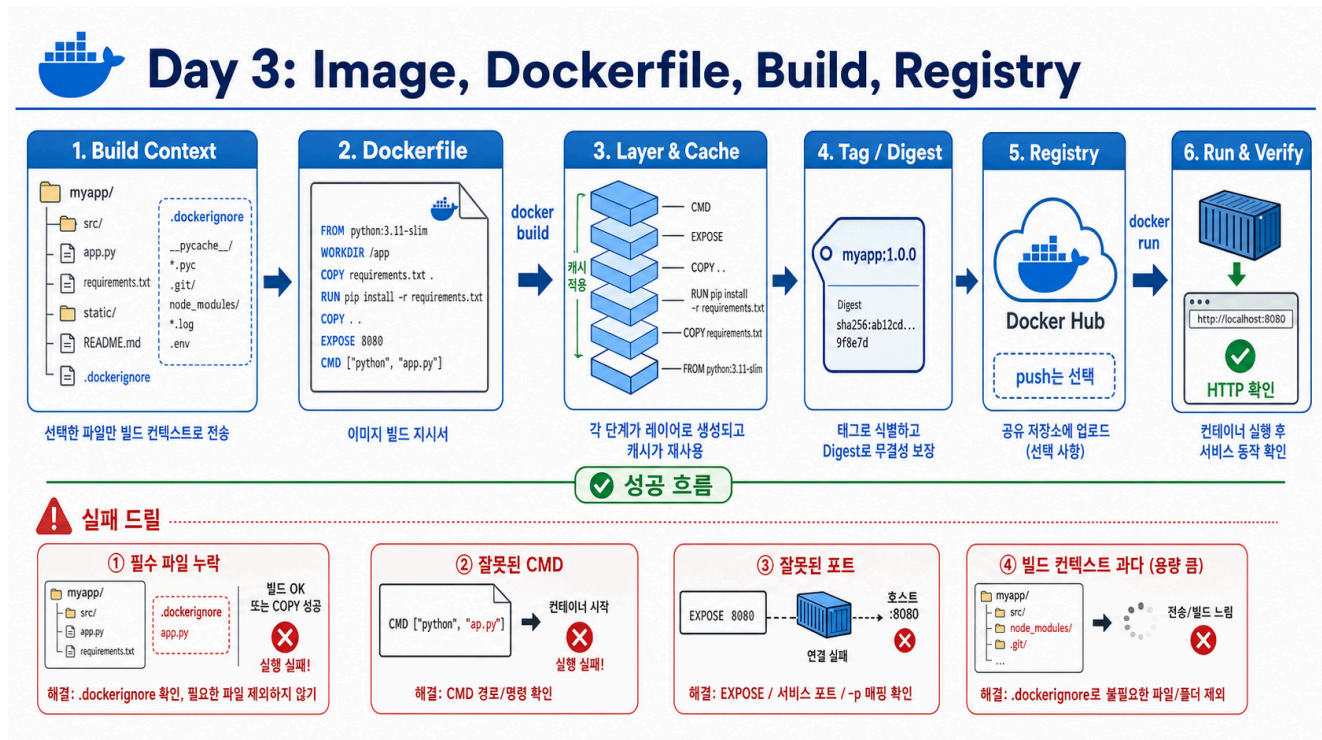


1교시: Image artifact 기준 잡기 - image, layer, tag, digest, registry



수업 목표

- image, container, artifact, layer, tag, digest, registry를 각각 다른 개념으로 설명한다.
- official image를 docker pull, docker images, docker history, docker image inspect로 읽는다.
- 오늘 만들 paperclip-static-site:day3 image의 납품 기준을 정한다.

왜 이 교시가 필요한가

Day 1~2에서는 이미 만들어진 official image를 가져와서 container를 실행했다. 학생 입장에서는 docker run postgres나 docker run nginx가 마법처럼 보일 수 있다. Day 3는 그 마법을 풀어 보는 날이다. 누군가 만든 image를 가져다 쓰는 수준에서 멈추지 않고, 우리가 가진 앱 파일을 image로 만들고, 그 image가 어떤 기준으로 실행 가능한 산출물인지 설명해야 한다.

여기서 중요한 것은 명령어 암기가 아니다. docker pull, docker images, docker history, docker inspect는 각각 다른 질문에 답한다.

```
docker pull      : registry에서 image를 가져온다.
docker images   : local에 어떤 image reference가 있는지 본다.
```

```
docker history : image가 어떤 layer 흐름으로 만들어졌는지 본다.  
docker inspect : image metadata, tag, digest, architecture, size를 본다.
```

요소별 설명

1. Image

Image는 container를 만들기 위한 읽기 전용 실행 재료다. 운영 관점에서는 앱 코드 + 실행에 필요한 파일 + 기본 실행 명령 + 기본 port 정보가 포장된 artifact에 가깝다. image 자체는 실행 중이 아니다. 실행하려면 `docker run`으로 container를 만들어야 한다.

예를 들어 `nginx:1.27-alpine` image는 nginx 실행에 필요한 파일과 기본 명령을 담고 있다. 우리가 오늘 만들 `paperclip-static-site:day3` image는 nginx base 위에 `index.html`, `styles.css`를 넣은 정적 웹앱 artifact가 된다.

2. Container

Container는 image로부터 만들어진 실행 중인 process다. 같은 image로 container를 여러 개 만들 수 있고, 각각은 다른 이름, 다른 host port, 다른 환경변수, 다른 volume을 가질 수 있다. 그래서 image 문제와 container 실행 조건 문제를 섞으면 안 된다.

Day 3에서는 image를 만드는 데 집중하고, Day 4에서는 같은 image를 다른 runtime config로 실행하며 `logs/inspect/exec/stats`를 확인한다.

3. Artifact

Artifact는 다른 사람이 재사용할 수 있게 남긴 결과물이다. 소스 폴더만 있으면 실행 환경이 사람마다 달라진다. 하지만 image tag, build 명령, run 명령, verify 결과, cleanup 방법이 함께 있으면 다른 사람이 같은 결과를 재현할 수 있다.

Day 3의 artifact 기준은 다음과 같다.

```
image tag: paperclip-static-site:day3  
build evidence: docker build 성공과 docker images 출력  
run evidence: container Up 상태  
verify evidence: HTTP/1.1 200 OK와 HTML 본문 확인  
handoff evidence: README에 build/run/check/cleanup 기록
```

4. Layer

Image는 한 덩어리 파일처럼 보이지만 내부적으로는 layer가 쌓인 구조다. Dockerfile의 `FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`, `EXPOSE` 같은 instruction은 image history에 흔적을 남긴다. layer를 보면 image가 어떤 과정을 거쳐 만들어졌는지 추적할 수 있다.

`docker history nginx:1.27-alpine`은 nginx official image가 어떤 layer 흐름을 가졌는지 보여준다. 나중에 우리가 만든 image에서 `COPY index.html`, `COPY styles.css` 흔적을 확인하면, 앱 파일이 image에 들어갔다는 증거가 된다.

5. Tag

Tag는 사람이 읽기 쉬운 image 이름표다. `nginx:1.27-alpine`에서 `nginx`는 repository 이름이고 `1.27-alpine`은 tag다. `paperclip-static-site:day3`에서 `day3`도 tag다.

중요한 점은 tag가 content 자체가 아니라는 것이다. tag는 바뀔 수 있다. `latest`처럼 느슨한 tag는 특히 재현성을 보장하지 않는다. 수업에서는 `day3`, `day3-v2`, `day3-reviewed`처럼 목적이 보이는 tag를 사용한다.

6. Digest

Digest는 registry에서 image content를 식별하는 값이다. tag가 사람이 읽기 쉬운 이름표라면 digest는 content를 더 강하게 식별하는 지문에 가깝다. 운영에서 재현성이 중요해질수록 tag만 보지 않고 digest도 함께 본다.

단, local에서 직접 build한 image는 아직 registry에 push/pull되지 않았기 때문에 `RepoDigests`가 비어 있을 수 있다. 이것은 곧바로 오류가 아니다. digest가 없네?가 아니라 이 image는 아직 registry 기준 content 식별자가 없을 수 있구나라고 해석해야 한다.

7. Registry

Registry는 image를 저장하고 공유하는 장소다. Docker Hub, AWS ECR 같은 서비스가 registry 역할을 한다. local image는 내 컴퓨터에만 있다. 다른 컴퓨터, GitHub Actions runner, Kubernetes cluster가 같은 image를 쓰려면 registry에 push되어 있어야 한다.

하지만 registry push는 무조건 좋은 일이 아니다. public repository에 secret이 포함된 image를 올리면 사고다. 그래서 Day 3에서는 push 자체보다 push gate를 먼저 가르친다.

- image context에 `.env/token/개인 파일`이 들어가지 않았는가?
- repository가 public인지 private인지 아는가?
- tag 이름이 수업/버전/용도를 설명하는가?
- credential이 README나 screenshot에 남지 않는가?

8. Run and verify

Image가 만들어졌다고 서비스가 정상이라는 뜻은 아니다. `docker run`으로 container를 실행하고, host에서 접근할 port를 열고, `curl`로 HTTP 응답을 확인해야 한다.

`docker ps`에서 Up은 process가 떠 있다는 뜻이다. `curl -I http://localhost:18083`에서 `HTTP/1.1 200 OK`가 나와야 사용자가 접근 가능한 정상 상태라고 말할 수 있다.

실습 명령

```
docker pull nginx:1.27-alpine
docker images nginx
docker history nginx:1.27-alpine
docker image inspect nginx:1.27-alpine --format "{{.Id}} {{.Architecture}}
{{.Size}}"
docker image inspect nginx:1.27-alpine --format "{{json .RepoTags}} {{json
.RepoDigests}}"
```

기대 결과와 해석

docker images nginx의 기대 패턴:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	1.27-alpine	<image-id>	<time>	<size>

해석: local image store에 nginx:1.27-alpine reference가 생겼다.

docker history nginx:1.27-alpine의 기대 패턴:

IMAGE	CREATED BY	SIZE
<layer>	CMD ["nginx" "-g" "daemon off;"]	0B
<layer>	EXPOSE map[80/tcp:{}]	0B
...		

해석: image가 layer 흐름을 가지고 있고, nginx가 foreground로 실행되는 기본 명령과 80번 container port 정보를 가진다.

docker image inspect의 기대 패턴:

```
sha256:<id> amd64 <size>
["nginx:1.27-alpine"] ["nginx@sha256:..."]
```

해석: image ID, architecture, size, tag, digest를 metadata로 확인했다.

학생이 말할 수 있어야 하는 문장

Image는 container의 실행 재료이고, container는 image로 만든 실행 중 process다.
 Layer는 image가 만들어진 흔적이고, tag는 사람이 읽는 이름표다.
 Digest는 registry 기준 content 식별자이며, local build image에서는 비어 있을 수 있다.

Registry는 image를 다른 runner나 cluster가 pull할 수 있게 공유하는 장소다.
Build 성공만으로 끝나지 않고 run과 HTTP verify까지 해야 납품 기준을 통과한다.

핵심 포인트

1교시는 명령어 준비 운동이 아니다. Day 3 전체에서 계속 사용할 언어를 맞추는 시간이다. image, layer, tag, digest, registry, run, verify를 구분하지 못하면 Dockerfile build와 push gate도 전부 흐릿해진다.

다음 연결

다음 교시는 이 artifact를 직접 만드는 계약서인 Dockerfile을 줄 단위로 읽는다. FROM, WORKDIR, COPY, EXPOSE, CMD가 각각 어떤 운영 약속을 담는지 확인한다.